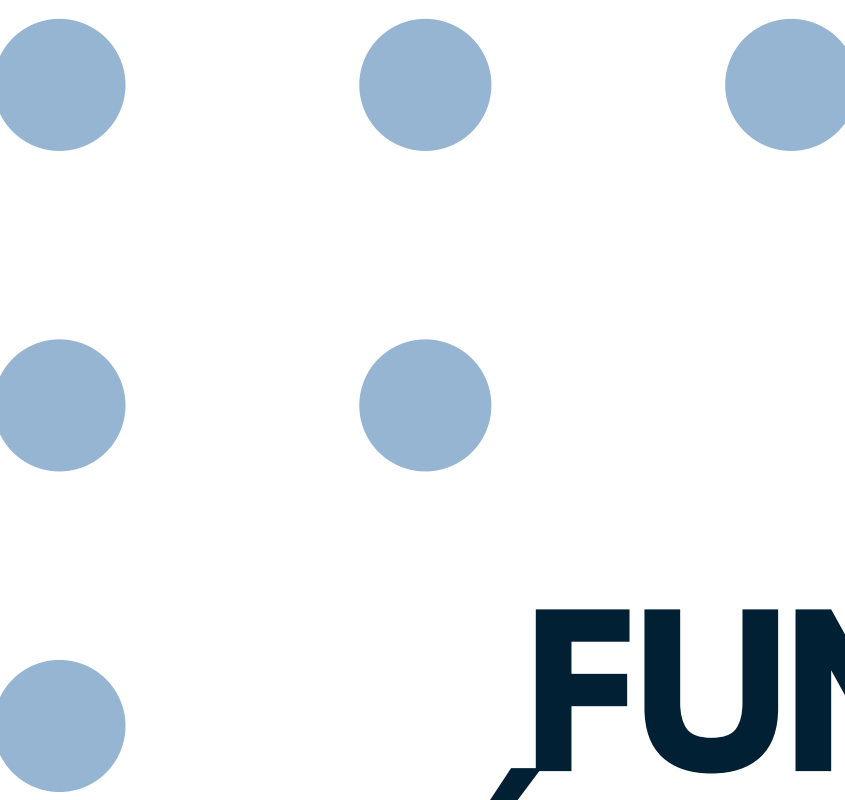
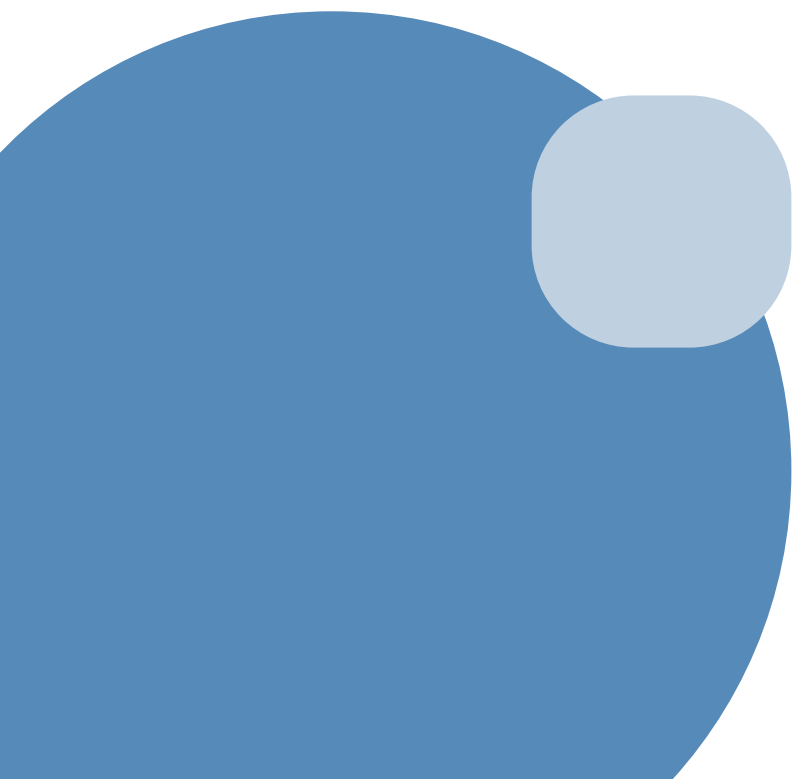




FUNDAMENTOS DE R PARA GRÁFICOS Y TABLAS CIENTÍFICAS

PROF. ASTRID LILIANA VARGAS SANCHEZ
M.S.C EN BIOINFORMÁTICA
UNIVERSIDAD INTERNACIONAL DE LA RIOJA





MODULO 1

INTRODUCCIÓN A R Y MANIPULACIÓN DE DATOS CON DPLYR

LENGUAJE DE PROGRAMACIÓN R



- Lenguaje de programación para análisis estadístico y visualización.
- Fue creado por Ross Ihaka y Robert Gentleman en 1996.
- Popular en bioestadística, bioinformática y ciencia de datos.
- Es software libre y gratuito
- Línea de comandos
- Se puede utilizar sin RStudio

INSTALACIÓN DE R

Descarga desde <https://cran.r-project.org/>



CRAN
[Mirrors](#)
[What's new?](#)
[Search](#)
[CRAN Team](#)

About R
[R Homepage](#)
[The R Journal](#)

Software
[R Sources](#)
[R Binaries](#)
[Packages](#)
[Task Views](#)
[Other](#)

Documentation
[Manuals](#)
[FAQs](#)
[Contributed](#)

Donations
[Donate](#)

The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- [Download R for Linux \(Debian, Fedora/Redhat, Ubuntu\)](#)
- [Download R for macOS](#)
- [Download R for Windows](#)

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Source Code for all Platforms

Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2025-04-11, How About a Twenty-Six) [R-4.5.0.tar.gz](#), read [what's new](#) in the latest version.
- The CRAN directory [src/base-prerelease](#) contains R alpha, beta, and rc releases as daily snapshots in time periods before a planned release.
- Between releases, the same directory [src/base-prerelease](#) contains snapshots of current patched and development versions. Please read about [new features and bug fixes](#) before filing corresponding feature requests or bug reports.
- Alternatively, daily snapshots are [available here](#).
- Source code of older versions of R is [available here](#).
- Contributed extension [packages](#).

Questions About R

- If you have questions about R like how to download and install the software, or what the license terms are, please read our [answers to frequently asked questions](#) before you send an email.

Supporting CRAN

- CRAN operations, most importantly hosting, checking, distributing, and archiving of R add-on packages for various platforms, crucially rely on technical, emotional, and financial support by the R community.

Please consider making [financial contributions](#) to the R Foundation for Statistical Computing.

What are R and CRAN?

R is 'GNU S', a freely available language and environment for statistical computing and graphics which provides a wide variety of statistical and graphical techniques: linear and nonlinear modelling, statistical tests, time series

R STUDIO (POSIT RSTUDIO)



- Entorno de desarrollo integrado (IDE).
- Interfaz amigable que facilita la escritura de código, la visualización de gráficos, la administración de proyectos y la depuración de scripts.
- Mejora la experiencia de programación en R, pero no es un lenguaje en sí mismo.
- Incluye una consola, un editor de scripts, un visor de gráficos y pestañas para explorar variables, archivos y paquetes.

INSTALACIÓN DE R STUDIO

Descarga desde: <https://posit.co/download/rstudio-desktop/>



PRODUCTS ▾

OPEN SOURCE ▾

USE CASES ▾

PARTNERS ▾

LEARN & SUPPORT ▾

ABOUT ▾



DOWNLOAD

RStudio Desktop

Used by millions of people weekly, the RStudio integrated development environment (IDE) is a set of tools built to help you be more productive with R and Python.

Don't want to download or install anything? Get started with RStudio on [Posit Cloud for free](#). If you're a professional data scientist looking to download RStudio and also need common enterprise features, don't hesitate to [book a call with us](#).

Want to learn about core or advanced workflows in RStudio? Explore the [RStudio User Guide](#) or the [Getting Started](#) section.

1: Install R

RStudio requires R 3.6.0+. Choose a version of R that matches your computer's operating system.

R is not a Posit product. By clicking on the link below to download and install R, you are leaving the Posit website.

2: Install RStudio

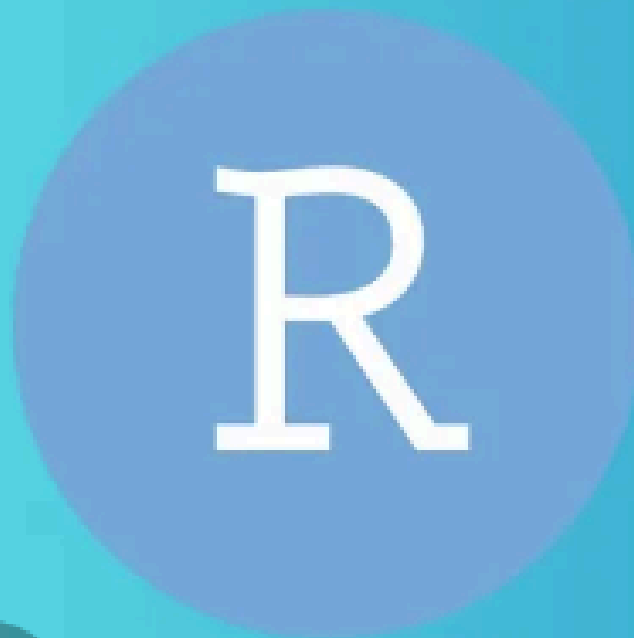
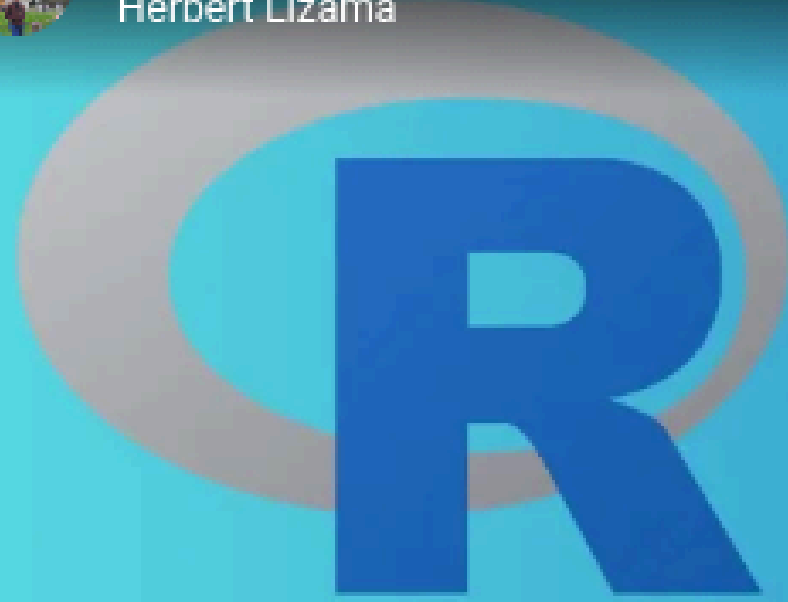
DOWNLOAD RSTUDIO DESKTOP FOR WINDOWS

Size: 281.27 MB | [SHA-256: 9E6F68CA](#) | Version: 2025.05.0+496 |
Released: 2025-05-05



Cómo instalar R y RStudio en menos de 2 minutos - 2026

Herbert Lizama



COMO INSTALAR R Y RSTUDIO 2026

Watch on  YouTube

<https://www.youtube.com/watch?v=hbgzW3Cvda4>

Material del curso:

Link: <https://github.com/Liliana223/Fundamentos-de-R-para-gr-ficos-y-tablas-cient-ficas>

VARIABLES

Una variable es un espacio de memoria al que accedemos mediante un identificador.

Una variable permite asociar un valor u objeto a un identificador. En la memoria del ordenador habrá una zona donde se almacena el objeto asociado a la variable

Integer	Números enteros (1, 5, -5, 8, -23). Se debe asignar con la letra L.	Date	Fecha (día, mes y año)
Numeric	Números con decimales (3,2; -6,32; 4,12)	POSIX	Fecha, hora y huso horario
Logical	Datos booleanos: True o false.	NA	Valor no disponible
Character	Texto	NULL	Elemento vacío
Factor	Información categórica	Inf	Infinito

TIPOS DE VARIABLES

Variables cualitativas (categóricas) : Estas no se expresan con números. Representan cualidades, atributos o categorías.

Variables cuantitativas (numéricas): Son aquellas que se expresan con números y permiten hacer operaciones matemáticas.

Nominales
No tienen un orden lógico.
Solo identifican o clasifican.

Ejemplo: Género: femenino, masculino, no binario.

Ordinales
Sí tienen un orden lógico, pero sin una distancia numérica clara entre categorías.

Ejemplo: Puesto en una competencia: 1.º, 2.º, 3.º.

Discretas
Toman valores enteros y contables. No tienen decimales.

Ejemplo: Número de hijos.

Continuas
Toman valores infinitos dentro de un intervalo, incluyen decimales. Se obtienen mediante medición.

Ejemplo: Peso (62.3 kg).

OPERADORES

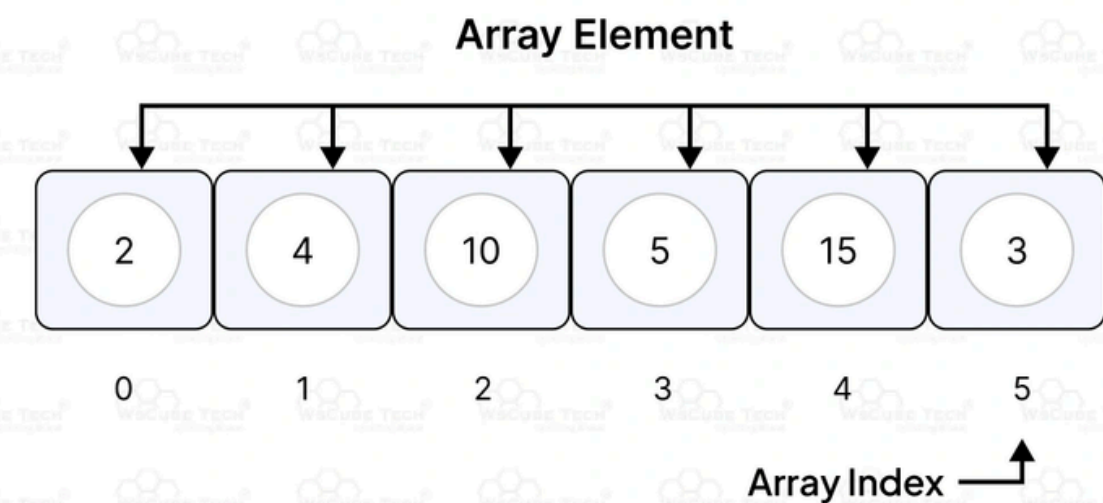
Un operador es un símbolo o conjunto de símbolos que indica a R que debe realizar una operación específica sobre uno o más valores (llamados operandos)

Tipo de Operador	Operador(es)	Descripción	Ejemplo
Aritméticos	+, -, *, /	Suma, resta, multiplicación, división	2 + 3 → 5
	^	Potencia	2^3 → 8
	%%	Resto de dividir un número por otro	10 %% 3 → 1
	%/%	División entera	10 %/% 3 → 3
Comparativos	==, !=	Igualdad, diferente	5 == 5 → TRUE
	<, >, <=, >=	Comparaciones	4 <= 3 → FALSE
Lógicos	&	Sentencia AND. Ambos elementos deben ser TRUE para que se obtenga TRUE	TRUE & FALSE → FALSE
		Sentencia OR. Al menos un elemento debe ser TRUE para que se obtenga TRUE	TRUE FALSE → TRUE
	!	Se invierte el dato booleano	!TRUE → FALSE
Asignación	<-, ->, =	Asignación de valores	x <- 10
Secuencia	:	Secuencia de números	1:5 → 1 2 3 4 5

ARRAY

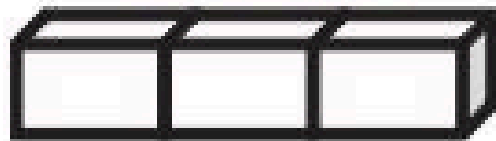
- Permite almacenar múltiples valores del mismo tipo en una sola variable, en lugar de declarar variables individuales para cada valor.
- Es una colección ordenada de elementos.
- Todos los elementos del array son del mismo tipo de dato (por ejemplo, enteros, cadenas, booleanos).
- Es posible el acceso a cada elemento de un array a través de un número entero que se denomina índice.
- La numeración de estos elementos dentro del array comienza en 0 (primer elemento del array) y finaliza en $n-1$ (último elemento del array) donde n es el tamaño completo de dicho array (Python, C, y JavaScript).
- En R la numeración de estos elementos dentro del array comienza en 1 (primer elemento del array) y finaliza en $n-1$ (último elemento del array) donde n es el tamaño completo de dicho array

Array Data Structure

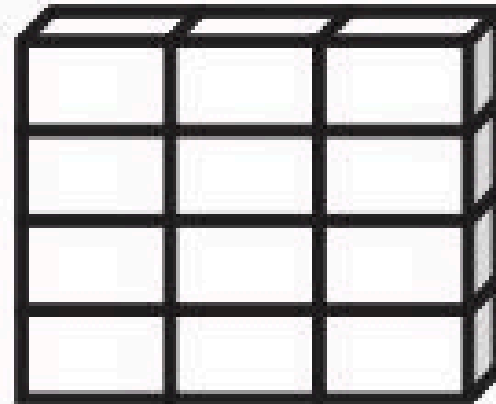


ESTRUCTURAS DE DATOS EN R

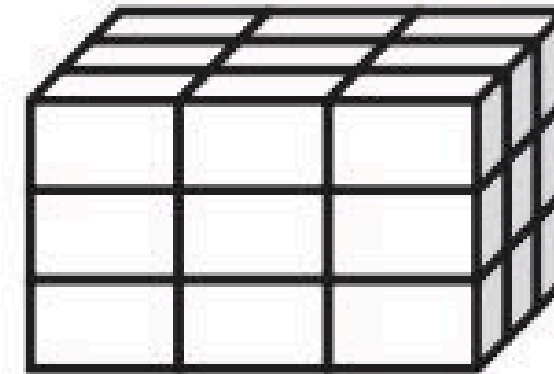
Vector



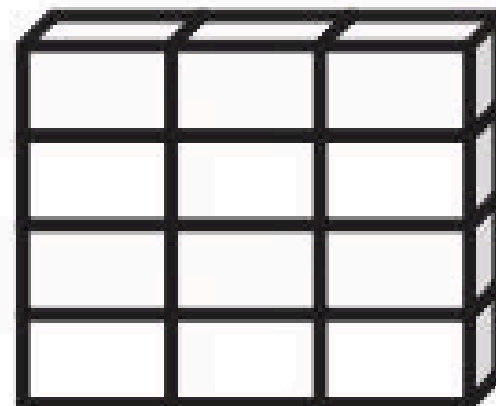
Matrix



Array



Data frame

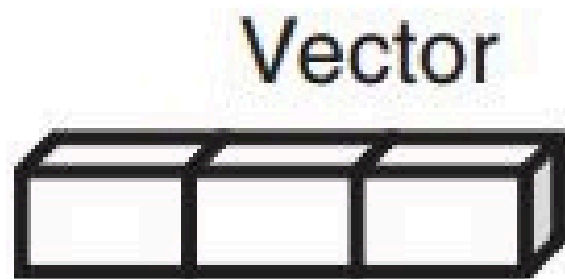


Columns can be different modes

List

Vectors
Arrays
Data frames
Lists

ESTRUCTURAS DE DATOS EN R

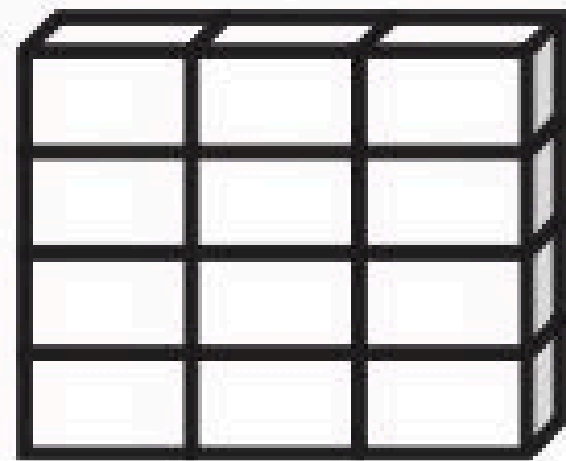


VECTOR

Array de una única
dimensión que almacena
datos del mismo tipo.

ESTRUCTURAS DE DATOS EN R

Matrix

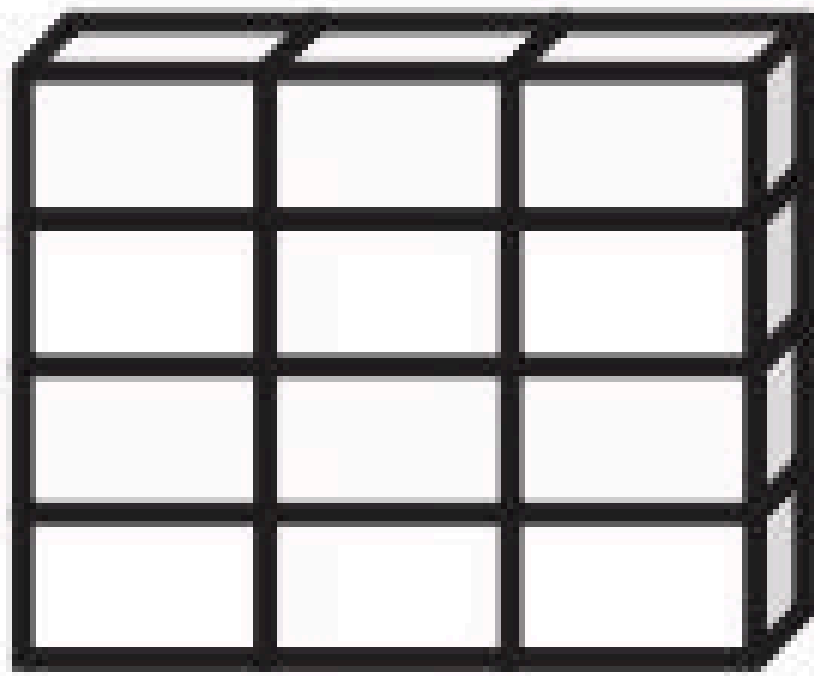


MATRIZ

Objeto de dos dimensiones
con formato tabla (Con filas
y columnas) con elementos
del mismo tipo.

ESTRUCTURAS DE DATOS EN R

Data frame



DATA FRAME

Objeto de dos dimensiones con formato tabla (Con filas y columnas) con elementos diferentes. Las filas en un data frame representan casos, individuos u observaciones, mientras que las columnas representan atributos, rasgos o variables. Las columnas deben tener la misma longitud

ESTRUCTURAS DE DATOS EN R

List

Vectors

Arrays

Data frames

Lists

LISTAS

Son estructuras que pueden contener elementos de diferentes tipos y estructuras: vectores, matrices, otros data frames o listas.

FUNCIÓN

Una función en R es un bloque de código reutilizable que realiza una o mas tareas de forma secuencial, nos permite organizar y automatizar procesos repetitivos. Puede tomar uno o más argumentos como entrada, ejecutar un conjunto de instrucciones, y devolver un resultado como salida.

`print()`

Toma como parámetro una expresión y muestra en la salida el resultado de evaluar la expresión.

`length()`

Devuelve el numero de elementos

Es un conjunto organizado de funciones y/o datos que han sido desarrollados para realizar tareas específicas, como análisis estadísticos, visualización de datos, manipulación de datos, etc. Permiten reutilizar código y facilitan tareas complejas sin tener que programar todo desde cero.



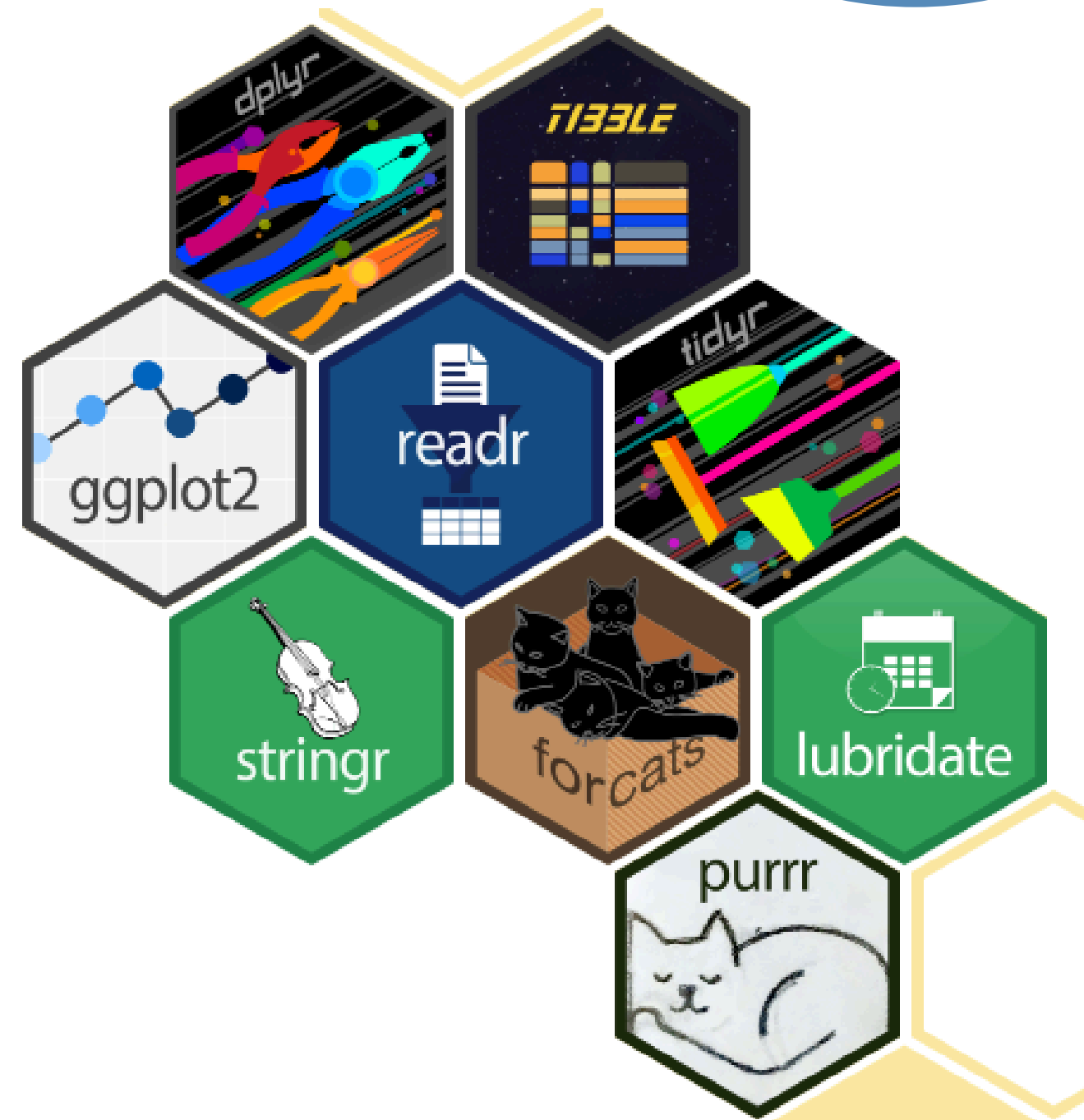
TIDYVERSE

Es un conjunto de paquetes en R diseñados para trabajar de forma coherente con datos "ordenados" (tidy data), facilitando tareas como importar, transformar, visualizar y modelar datos.

Tidyverse packages:

<https://www.tidyverse.org/packages/>

```
install.packages("tidyverse")  
library(tidyverse)
```



Paquete	Función principal	Uso típico
ggplot2	Visualización de datos con una gramática de gráficos.	Crear gráficos personalizados.
dplyr	Manipulación y transformación de datos.	Filtrar, seleccionar, agrupar, resumir.
tidyr	Organización de datos en formato ordenado (tidy).	Separar, unir, pivotear columnas.
readr	Importación rápida y limpia de datos desde archivos.	Leer CSV, TXT, TSV, etc.
tibble	Estructura moderna de data frames.	Imprime datos de forma clara y legible.
purrr	Programación funcional con listas y vectores.	Aplicar funciones a múltiples elementos.
stringr	Manipulación de texto (strings).	Buscar, reemplazar o modificar texto.
forcats	Manejo de factores (variables categóricas).	Reordenar o recodificar categorías.

READR



Paquete del tidyverse para leer archivos de datos



Permite importar datos de forma rápida y eficiente

Funciones principales

- ◆ `read_csv()` → archivos CSV
- ◆ `read_tsv()` → archivos tabulados
- ◆ `read_delim()` → archivos con separadores personalizados
- ◆ `write_csv()` → exportar datos

El paquete readr está diseñado para leer archivos de texto como:

TXT

TSV

CSV

Excel:

```
library(readxl)  
  
datos <- read_excel("datos.xlsx")
```



Data import with the tidyverse :: CHEATSHEET



Read Tabular Data with readr

```
read_* (file, col_names = TRUE, col_types = NULL, col_select = NULL, id = NULL, locale = Inf,
skip = 0, na = c("", "NA"), guess_max = min(1000, n_max), show_col_types = TRUE) See ?read_delim
```

 <pre>A B C 1 2 3 4 5 NA</pre>	→	<table border="1"><thead><tr><th>A</th><th>B</th><th>C</th></tr></thead><tbody><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	A	B	C	1	2	3	4	5	NA	read_delim ("file.txt", delim = " ") Read files with any delimiter. If no delimiter is specified, it will automatically guess. To make file.txt, run: write_file("A B C\n1 2 3\n4 5 NA", file = "file.txt")
A	B	C										
1	2	3										
4	5	NA										
 <pre>A,B,C 1,2,3 4,5,NA</pre>	→	<table border="1"><thead><tr><th>A</th><th>B</th><th>C</th></tr></thead><tbody><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	A	B	C	1	2	3	4	5	NA	read_csv ("file.csv") Read a comma delimited file with period decimal marks. write_file("A,B,C\n1,2,3\n4,5,NA", file = "file.csv")
A	B	C										
1	2	3										
4	5	NA										
 <pre>A;B;C 1;2;3 4;5;NA</pre>	→	<table border="1"><thead><tr><th>A</th><th>B</th><th>C</th></tr></thead><tbody><tr><td>1.5</td><td>2</td><td>3</td></tr><tr><td>4.5</td><td>5</td><td>NA</td></tr></tbody></table>	A	B	C	1.5	2	3	4.5	5	NA	read_csv2 ("file.csv") Read semicolon delimited files with comma decimal marks. write_file("A;B;C\n1,5;2,3\n4,5;5;NA", file = "file2.csv")
A	B	C										
1.5	2	3										
4.5	5	NA										
 <pre>A B C 1 2 3 4 5 NA</pre>	→	<table border="1"><thead><tr><th>A</th><th>B</th><th>C</th></tr></thead><tbody><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	A	B	C	1	2	3	4	5	NA	read_tsv ("file.tsv") Read a tab delimited file. Also read_table() . read_fwf ("file.tsv", fwf_widths(c(2, 2, NA))) Read a fixed width file. write_file("A\tB\tC\n1\t2\t3\n4\t5\tNA\n", file = "file.tsv")
A	B	C										
1	2	3										
4	5	NA										

One of the first steps of a project is to import outside data into R. Data is often stored in tabular formats, like csv files or spreadsheets.



The front page of this sheet shows how to import and save text files into R using **readr**.



The back page shows how to import spreadsheet data from Excel files using **readxl** or Google Sheets using **googlesheets4**.

OTHER TYPES OF DATA
Try one of the following packages to import other types of files:

- **haven** - SPSS, Stata, and SAS files
- **DBI** - databases
- **jsonlite** - json
- **xml2** - XML
- **httr** - Web APIs
- **rvest** - HTML (Web Scraping)
- **readr::read_lines()** - text data

Column Specification with readr

Column specifications define what data type each column of a file will be imported as. By default readr will generate a column spec when a file is read and output a summary.

spec(x) Extract the full column specification for the given imported data frame.

```
spec(x)
# cols(
#   age = col_integer(),
#   edu = col_character(),
#   earn = col_double()
# )
```

age is an integer

earn is a double (numeric)

edu is a character

USEFUL COLUMN ARGUMENTS

Hide col spec message
`read_* (file, show_col_types = FALSE)`

Select columns to import
Use names, position, or selection helpers.
`read_* (file, col_select = c(age, earn))`

Guess column types
To guess a column type, `read_* ()` looks at the first 1000 rows of data. Increase with **guess_max**.
`read_* (file, guess_max = Inf)`

COLUMN TYPES

Each column type has a function and corresponding string abbreviation.

- **col_logical()** - "l"
- **col_integer()** - "i"
- **col_double()** - "d"
- **col_number()** - "n"
- **col_character()** - "c"
- **col_factor(levels, ordered = FALSE)** - "f"
- **col_datetime(format = "")** - "T"
- **col_date(format = "")** - "D"
- **col_time(format = "")** - "t"
- **col_skip()** - "s", "._"
- **col_guess()** - "?"

DEFINE COLUMN SPECIFICATION

Set a default type

```
read_csv(
  file,
  col_type = list(default = col_double())
)
```

Use column type or string abbreviation

```
read_csv(
  file,
  col_type = list(x = col_double(), y = "i", z = "._")
)
```

Use a single string of abbreviations

```
# col types: skip, guess, integer, logical, character
read_csv(
  file,
  col_type = "._?ilc"
)
```

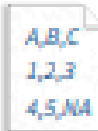
`install.packages("readr")`
`library(readr)`

USEFUL READ ARGUMENTS

<table border="1"><thead><tr><th>A</th><th>B</th><th>C</th></tr></thead><tbody><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	A	B	C	1	2	3	4	5	NA	No header <code>read_csv("file.csv", col_names = FALSE)</code>	<table border="1"><thead><tr><th>1</th><th>2</th><th>3</th></tr></thead><tbody><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	1	2	3	4	5	NA	Skip lines <code>read_csv("file.csv", skip = 1)</code>						
A	B	C																						
1	2	3																						
4	5	NA																						
1	2	3																						
4	5	NA																						
<table border="1"><thead><tr><th>x</th><th>y</th><th>z</th></tr></thead><tbody><tr><td>A</td><td>B</td><td>C</td></tr><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	x	y	z	A	B	C	1	2	3	4	5	NA	Provide header <code>read_csv("file.csv", col_names = c("x", "y", "z"))</code>	<table border="1"><thead><tr><th>A</th><th>B</th><th>C</th></tr></thead><tbody><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	A	B	C	1	2	3	4	5	NA	Read a subset of lines <code>read_csv("file.csv", n_max = 1)</code>
x	y	z																						
A	B	C																						
1	2	3																						
4	5	NA																						
A	B	C																						
1	2	3																						
4	5	NA																						
	Read multiple files into a single table <code>read_csv(c("f1.csv", "f2.csv", "f3.csv"), id = "origin_file")</code>	<table border="1"><thead><tr><th>A</th><th>B</th><th>C</th></tr></thead><tbody><tr><td>NA</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	A	B	C	NA	2	3	4	5	NA	Read values as missing <code>read_csv("file.csv", na = c("1"))</code>												
A	B	C																						
NA	2	3																						
4	5	NA																						
 <pre>A;B;C 1,5;2,3,0</pre>	Specify decimal marks <code>read_delim("file2.csv", locale = locale(decimal_mark = ","))</code>																							

Save Data with readr

```
write_* (x, file, na = "NA", append, col_names, quote, escape, eol, num_threads, progress)
```

<table border="1"><thead><tr><th>A</th><th>B</th><th>C</th></tr></thead><tbody><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>NA</td></tr></tbody></table>	A	B	C	1	2	3	4	5	NA	→	 <pre>A,B,C 1,2,3 4,5,NA</pre>	write_delim (x, file, delim = ",") Write files with any delimiter.
A	B	C										
1	2	3										
4	5	NA										
			write_csv (x, file) Write a comma delimited file.									
			write_csv2 (x, file) Write a semicolon delimited file.									
			write_tsv (x, file) Write a tab delimited file.									

<https://www.kaggle.com/datasets/samybaladram/iris-dataset-extended/data>

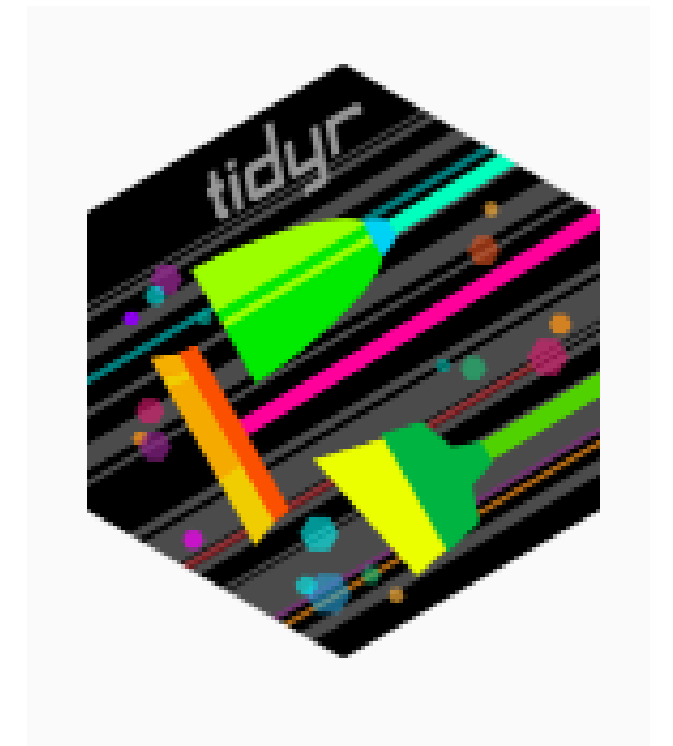
TIDYR

 Paquete del tidyverse para organizar datos (cambiar la forma (estructura) de los datos)

 Permite transformar datos a formato "tidy"

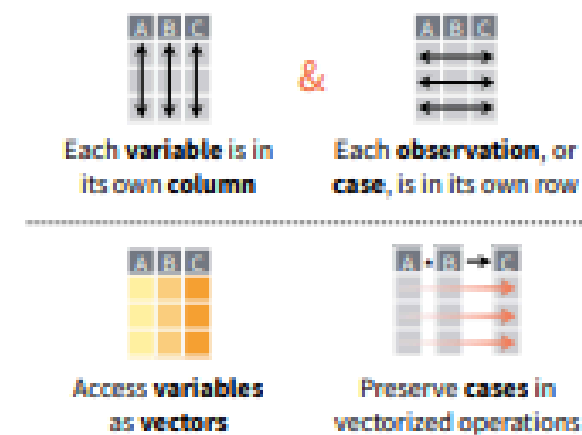
Funciones principales

- ♦ `pivot_longer()` → formato ancho → largo
- ♦ `pivot_wider()` → formato largo → ancho
- ♦ `separate()` → dividir columnas
 - ♦ `unite()` → unir columnas



Data tidying with tidyr :: CHEATSHEET

Tidy data is a way to organize tabular data in a consistent data structure across packages. A table is tidy if:



Tibbles

AN ENHANCED DATA FRAME

Tibbles are a table format provided by the **tibble** package. They inherit the data frame class, but have improved behaviors:

- **Subset** a new tibble with `[],` a vector with `[[` and `$.`
- **No partial matching** when subsetting columns.
- **Display** concise views of the data on one screen.

`options(tibble.print_max = n, tibble.print_min = m, tibble.width = Inf)` Control default display settings.

`View()` or `glimpse()` View the entire data set.

CONSTRUCT A TIBBLE

`tibble(...)` Construct by columns.

`tibble(x = 1:3, y = c("a", "b", "c"))`

`tribble(...)` Construct by rows.

`tribble(~x, ~y,
1, "a",
2, "b",
3, "c")`

Both make this tibble

A tibble: 3 x 2
x y
1 a
2 b
3 c

`as_tibble(x, ...)` Convert a data frame to a tibble.

`enframe(x, name = "name", value = "value")`

Convert a named vector to a tibble. Also `deframe()`.

`is_tibble(x)` Test whether x is a tibble.

Reshape Data

Pivot data to reorganize values into a new layout.

table4a

country	1999	2000
A	0.7K	2K
B	37K	80K
C	213K	213K

→

country	year	cases
A	1999	0.7K
B	1999	37K
C	1999	213K
A	2000	2K
B	2000	80K
C	2000	213K

`pivot_longer(data, cols, names_to = "name", values_to = "value", values_drop_na = FALSE)`

"Lengthen" data by collapsing several columns into two. Column names move to a new `names_to` column and values to a new `values_to` column.

`pivot_longer(table4a, cols = 2:3, names_to = "year", values_to = "cases")`

table2

country	year	type	count
A	1999	cases	0.7K
A	1999	pop	19M
A	2000	cases	2K
A	2000	pop	20M
B	1999	cases	37K
B	1999	pop	172M
B	2000	cases	80K
B	2000	pop	174M
C	1999	cases	213K
C	1999	pop	1T
C	2000	cases	213K
C	2000	pop	1T

→

country	year	cases	pop
A	1999	0.7K	19M
A	2000	2K	20M
B	1999	37K	172M
B	2000	80K	174M
C	1999	213K	1T
C	2000	213K	1T

`pivot_wider(data, names_from = "name", values_from = "value")`

The inverse of `pivot_longer()`. "Widen" data by expanding two columns into several. One column provides the new column names, the other the values.

`pivot_wider(table2, names_from = type, values_from = count)`

Split Cells

Use these functions to split or combine cells into individual, isolated values.

table5

country	century	year
A	19	99
A	20	00
B	19	99
B	20	00

→

country	year
A	1999
A	2000
B	1999
B	2000

`unite(data, col, ..., sep = "_", remove = TRUE, na.rm = FALSE)` Collapse cells across several columns into a single column.

`unite(table5, century, year, col = "year", sep = "")`

table3

country	year	rate
A	1999	0.7K/19M
A	2000	2K/20M
B	1999	37K/172M
B	2000	80K/174M

→

country	year	cases	pop
A	1999	0.7K	19M
A	2000	2K	20M
B	1999	37K	172
B	2000	80K	174

`separate_wider_delim(data, cols, delim, ..., names = NULL, names_sep = NULL, names_repair = "check_unique", too_few, too_many, cols_remove = TRUE)` Separate each cell in a column into several columns. Also `separate_wider_regex()` and `separate_wider_position()`.

`separate(table3, rate, sep = "/")`
`into = c("cases", "pop")`

table3

country	year	rate
A	1999	0.7K
A	1999	19M
A	2000	2K
A	2000	20M
B	1999	37K
B	1999	172M
B	2000	80K
B	2000	174M

→

country	year	rate
A	1999	0.7K/19M
A	2000	2K/20M
B	1999	37K/172M
B	2000	80K/174M

`separate_longer_delim(data, cols, delim, ..., width, keep_empty)` Separate each cell in a column into several rows.

`separate_longer_delim(table3, rate, sep = "/")`

Expand Tables

Create new combinations of variables or identify implicit missing values (combinations of variables not present in the data).

x

col1	col2	col3
A	1	3
B	1	4
B	2	2

→

col1	col2	col3
A	1	3
A	2	NA
B	1	4
B	2	2

`expand(data, ...)` Create a new tibble with all possible combinations of the values of the variables listed in ... Drop other variables.
`expand(mtcars, cyl, gear, carb)`

x

col1	col2	col3
A	1	3
B	1	4
B	2	2

→

col1	col2	col3
A	1	3
A	2	NA
B	1	4
B	2	2

`complete(data, ..., fill = list())` Add missing possible combinations of values of variables listed in ... Fill remaining variables with NA.
`complete(mtcars, cyl, gear, carb)`

Handle Missing Values

Drop or replace explicit missing values (NA).

x

col1	col2
A	1
B	NA
C	NA
D	3
E	NA

→

col1	col2
A	1
D	3

`drop_na(data, ...)` Drop rows containing NA's in ... columns.
`drop_na(x, x2)`

x

col1	col2
A	1
B	NA
C	NA
D	3
E	NA

→

col1	col2
A	1
B	1
C	1
D	3
E	3

`fill(data, ..., .direction = "down")` Fill in NA's in ... columns using the next or previous value.
`fill(x, x2)`

x

col1	col2
A	1
B	NA
C	NA
D	3
E	NA

→

col1	col2
A	1
B	2
C	2
D	3
E	2

`replace_na(data, replace)` Specify a value to replace NA in selected columns.
`replace_na(x, list(x2 = 2))`

`install.packages("tidyr")`
`library(tidyr)`

PIVOT_LONGER() # CONVERTIR DE FÓRMATO ANCHO A LARGO

Formato ancho

especie	longitud_sepalo	ancho_sepalo
setosa	5.1	3.5
versicolor	6.0	2.8

- ✓ Las columnas se volvieron filas
- ✓ Se crearon dos nuevas columnas:
variable → nombre original
valor → datos

Formato largo

especie	variable	valor
setosa	longitud_sepalo	5.1
setosa	ancho_sepalo	3.5
versicolor	longitud_sepalo	6.0
versicolor	ancho_sepalo	2.8

- ✓ Facilita gráficos con ggplot2
- ✓ Permite análisis más flexibles

DPLYR



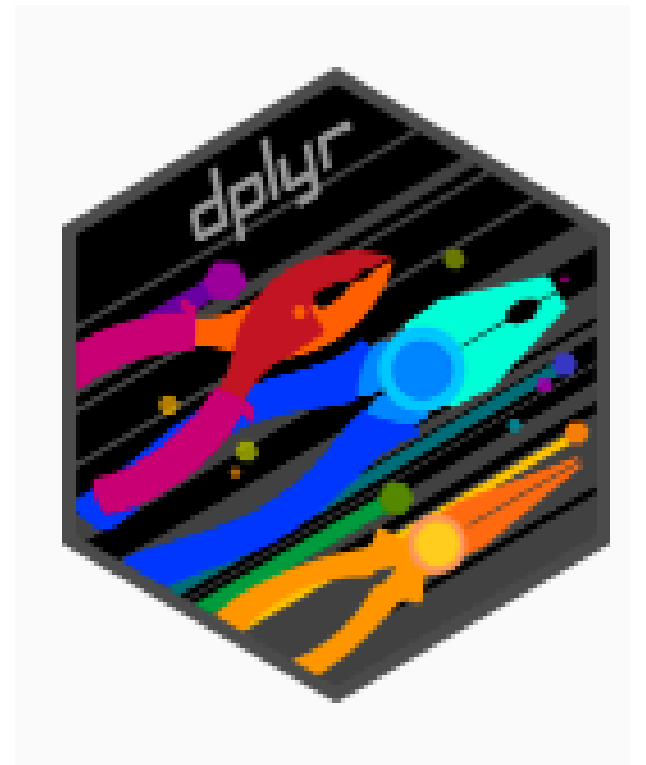
Paquete del tidyverse para transformar y manipular datos (modificar el contenido de los datos)



Permite trabajar con tablas de forma rápida y clara

Funciones principales

- ♦ `select()` → seleccionar columnas
 - ♦ `filter()` → filtrar filas
 - ♦ `mutate()` → crear variables
 - ♦ `arrange()` → ordenar datos
- ♦ `group_by() + summarise()` → agrupar y resumir



Domar Datos con dplyr and tidyr

Hoja de Referencia



Sintaxis - Convenciones útiles para la doma

`dplyr::tbl_df(iris)`

Convierte datos a una *tbl*. Objetos *tbl* son mas fáciles de inspeccionar que *data frames*. Ro solo muestra los datos que caben en la pantalla:

```
Source: local data frame [150 x 5]
  Sepal.Length Sepal.Width Petal.Length
1           5.1         3.5          1.4
2           4.9         3.0          1.4
3           4.7         3.2          1.3
4           4.6         3.1          1.5
5           5.0         3.6          1.4
Variables not shown: Petal.Width (dbl),
Species (fctr)
```

`dplyr::glimpse(iris)`

Resumen con mucha información sobre los datos *tbl*.

`utils::View(iris)`

Observa el conjunto de datos en lo que parece una hoja de cálculo (nota la V mayúscula).

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.8	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa

`dplyr::%>%`

Pasa el objeto a la izquierda al primer argumento (o argumento .) de la función a la derecha creando un tubo.

`x %>% f(y)` es lo mismo que `f(x, y)`

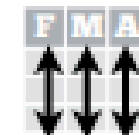
`y %>% f(x, ., z)` es lo mismo que `f(x, y, z)`

El uso de tubos y tuberías (en Ingles: "Piping") con `%>%` resulta en código mas legible. Por ejemplo:

```
iris %>%
  group_by(Species) %>%
```

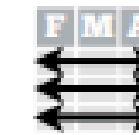
Datos Ordenados - La base para domar datos en R

En un
conjunto de
datos
ordenado:



Cada **variable** tiene
su propia **columna**

&

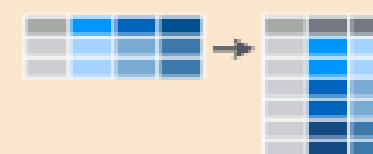


Cada **observación**
tiene su propia **fila**

Datos ordenados complementan las **operaciones vectorizadas** de R. Automáticamente R preserva observaciones mientras manipulas las variables. No hay otro formato que funciona tan intuitivamente en R.



Remodelar Datos - Cambia el esquema de los datos



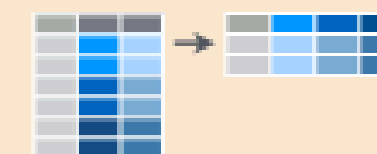
`tidyr::gather(cases, "year", "n", 2:4)`

Reune columnas en filas.



`tidyr::separate(storms, date, c("a", "m", "d"))`

Separa una columna en varias.



`tidyr::spread(pollution, size, amount)`

Extiende filas en columnas.



`tidyr::unite(data, col, ..., sep)`

Une varias columnas en una.

`dplyr::data_frame(a = 1:3, b = 4:6)`

Combina vectores en un *data frame* (optimizado).

`dplyr::arrange(mtcars, mpg)`

Ordena filas por valores de una columna (bajo a alto).

`dplyr::arrange(mtcars, desc(mpg))`

Ordena filas por valores de una columna (alto a bajo).

`dplyr::rename(tb, y = year)`

Cambia el nombre de columnas de un *data frame*.

Subconjuntos de Observaciones



`dplyr::filter(iris, Sepal.Length > 7)`

Extrae filas que cumplen criterios lógicos.

`dplyr::distinct(iris)`

Remueve filas duplicadas.

`dplyr::sample_frac(iris, 0.5, replace = TRUE)`

Selecciona una fracción de filas al azar.

`dplyr::sample_n(iris, 10, replace = TRUE)`

Selecciona n filas al azar.

`dplyr::slice(iris, 10:15)`

Selecciona filas por posición.

`dplyr::top_n(storms, 2, date)`

Selecciona y ordena las n entradas mas altas (por grupo si los datos estan agrupados).

Subconjuntos de Variables



`dplyr::select(iris, Sepal.Width, Petal.Length, Species)`

Selecciona columnas por nombre o funciones de ayuda.

Funciones de ayuda para for select - ?select

```
select(iris, contains("."))
#> Selecciona columnas cuyos nombres contienen una cadena de caracteres.

select(iris, ends_with("Length"))
#> Selecciona columnas cuyos nombres terminan con una cadena de caracteres.

select(iris, everything())
#> Selecciona todas las columnas.

select(iris, matches("1."))
#> Selecciona columnas cuyo nombre cumple con una expresión regular.

select(iris, num_range("x", 1:5))
#> Selecciona columna con nombres x1, x2, x3, x4, x5.

select(iris, one_of(c("Species", "Genus")))
#> Selecciona columnas cuyos nombres están en un grupo de nombres.

select(iris, starts_with("Sepal"))
#> Selecciona columnas cuyos nombres comienzan con una cadena de caracteres.

select(iris, Sepal.Length:Petal.Width)
#> Selecciona todas las columnas entre Sepal.Length and Petal.Width (incluyente).

select(iris, -Species)
#> Selecciona todas las columnas excepto Species.
```

```
install.packages("dplyr")
library(dplyr)
```

Transformar un dataset crudo en un formato limpio, organizado y adecuado para generar visualizaciones y tablas claras, reproducibles y listas para publicación.

- ✓ Cada variable → una columna
- ✓ Cada observación → una fila
- ✓ Cada valor → una celda

Preparación de datos para gráficos y tablas científicas



Limpieza de datos

Eliminar valores faltantes y corregir formatos



Selección de variables

Elegir solo la información relevante



Filtrado de datos

Analizar grupos específicos



Creación de variables

Generar nuevas métricas (ej: IMC)



Resumen de datos

Calcular estadísticas clave



Transformación (tidy)

Organizar datos para visualización



Orden y etiquetado

Definir categorías y nombres claros

✨ Datos bien preparados → Gráficos claros y tablas publicables ✨

ELIMINAR VALORES FALTANTES

Estamos eliminando todas las filas que tienen al menos un valor faltante (NA).

Antes:

especie	longitud_sepalo	ancho_sepalo
setosa	5.1	3.5
versicolor	NA	2.8
virginica	6.5	3.0

Después de `drop_na()` :

especie	longitud_sepalo	ancho_sepalo
setosa	5.1	3.5
virginica	6.5	3.0

- ✓ Cuando hay pocos valores faltantes
- ✓ Cuando no afectan mucho el dataset
- ✓ Cuando necesitas datos completos

REEMPLAZAR VALORES FALTANTES. REPLACE_NA

En lugar de eliminar filas, estamos rellendo los valores faltantes (NA) con un valor específico.

Antes:

area_petal	area_sepalo
0.42	17.6
NA	15.2
1.20	NA

Después:

area_petal	area_sepalo
0.42	17.6
0	15.2
1.20	0

REEMPLAZAR VALORES FALTANTES. FILL()

Estamos rellendo los valores faltantes (NA) usando valores anteriores de la misma columna.

tipo_suelo

arenoso

NA

NA

arcilloso

tipo_suelo

arenoso

arenoso

arenoso

arcilloso

El valor para reemplazar un NA debe tener sentido con el tipo de variable y el contexto del estudio.

Variable	Recomendación
area_petal	media o por grupo
area_sepal	media o mediana
tipo_suelo	"desconocido"
especie	mejor NO reemplazar ⚠

VERIFICAR DUPLICADOS

Estamos identificando filas repetidas en el dataset.

especie	longitud_sepalo
setosa	5.1
setosa	5.1
virginica	6.5

FALSE → fila única

TRUE → fila repetida

FALSE
TRUE
FALSE



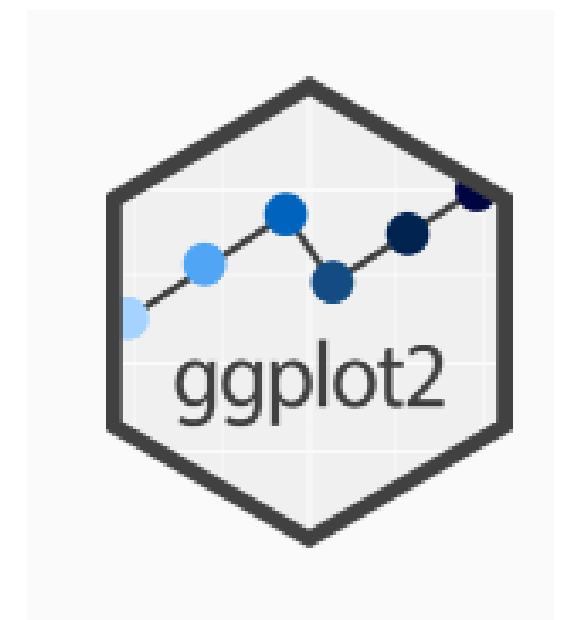
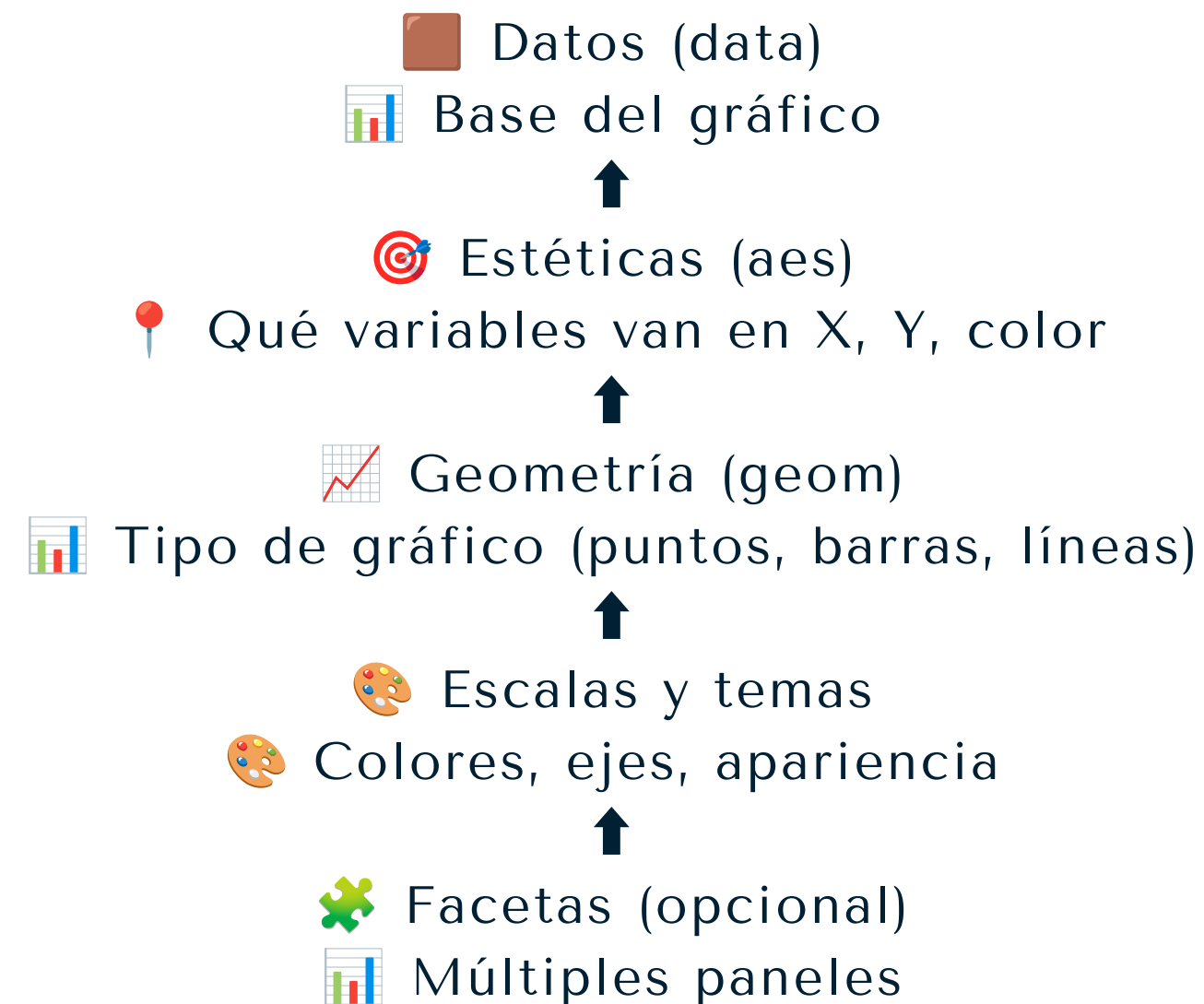
MODULO 2

GRÁFICOS CON GGLOT2 LISTOS PARA PUBLICAR

GGPLOT2

 Paquete para crear gráficos en R

- ◆ Permite construir gráficos por capas
- ◆ Flexible y altamente personalizable



Visualización de Datos

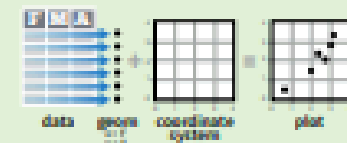
usando ggplot2

Guía Rápida

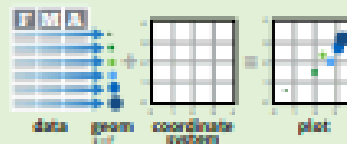


Conceptos Básicos

ggplot2 se basa en la idea que cualquier gráfica se puede construir usando estos tres componentes: **datos**, **coordenadas** y **objetos geométricos (geoms)**. Este concepto se llama: **gramática de las gráficas**.



Para visualizar resultados, asigne variables a las propiedades visuales, o **estéticas**, como **tamaño**, **color** y **posición x ó y**.



Para construir una gráfica complete este patrón

```
ggplot(data = <DATOS>) +  
  <FUNCION_GEOM> (  
    mapping = aes() (<ESTETICAS>),  
    stat = <STAT>,  
    position = <POSICION>  
  ) +  
  <FUNCION_COORDINADAS> +  
  <FUNCION_FACETA> +  
  <FUNCION_ESCALA> +  
  <FUNCION_TEMA>
```

Requerido

No Requerido, se proveen valores iniciales

`ggplot(data = mpg, aes(x = cty, y = hwy))`

Este comando comienza una gráfica, complétela mediante agregando capas, un **geom** por capa.

estéticas datos geom

`qplot(x = cty, y = hwy, data = mpg, geom = "point")`
Este comando es una gráfica completa, tiene datos, las estéticas están asignadas y por lo menos un geom.

`last_plot()`

Devuelve la última gráfica

`ggsave("plot.png", width = 5, height = 5)`

La última gráfica es grabada como una imagen de 5 por 5 pulgs., usa el mismo tipo de archivo que la extensión

Geoms

Funciones geom se utilizan para visualizar resultados. Asigne variables a las propiedades estéticas del geom. Cada geom forma una capa.

Geométricas Elementales

`a <- ggplot(economics, aes(date, unemploy))`
`b <- ggplot(seals, aes(x = long, y = lat))`



`a + geom_blank()`
(Bueno para expandir límites)



`b + geom_curve(aes(yend = lat + 1,
xend = long + 1, curvature = z))` - x, xend, y, yend,
alpha, angle, color, curvature, linetype, size



`a + geom_path(linetype = "solid",
linejoin = "round", linemitre = 1)`
x, y, alpha, color, group, linetype, size



`a + geom_polygon(aes(group = group))`
x, y, alpha, color, fill, group, linetype, size



`b + geom_rect(aes(xmin = long, ymin = lat,
xmax = long + 1, ymax = lat + 1))` - xmax, xmin,
ymax, ymin, alpha, color, fill, linetype, size



`a + geom_ribbon(aes(ymin = unemploy - 900,
ymax = unemploy + 900))` - x, ymax, ymin,
alpha, color, fill, group, linetype, size

Segmentos Lineales

propiedades básicas: x, y, alpha, color, linetype, size



`b + geom_abline(aes(intercept = 0, slope = 1))`

`b + geom_hline(aes(yintercept = lat))`

`b + geom_vline(aes(xintercept = long))`



`b + geom_segment(aes(yend = lat + 1, xend = long + 1))`



`b + geom_spoke(aes(angle = 1:1155, radius = 1))`

Una Variable

Continua

`c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)`



`c + geom_area(stat = "bin")`
x, y, alpha, color, fill, linetype, size



`c + geom_density(kernel = "gaussian")`
x, y, alpha, color, fill, group, linetype, size, weight



`c + geom_dotplot()`
x, y, alpha, color, fill



`c + geom_freqpoly()`
x, y, alpha, color, group, linetype, size



`c + geom_histogram(binwidth = 5)`
x, y, alpha, color, fill, linetype, size, weight



`c2 + geom_qq(aes(sample = hwy))`
x, y, alpha, color, fill, linetype, size, weight

Discreta



`d <- ggplot(mpg, aes(R))`
`d + geom_bar()`
x, alpha, color, fill, linetype, size, weight

Dos Variables

X Continua, Y Continua

`e <- ggplot(mpg, aes(cty, hwy))`



`e + geom_label(aes(label = cty), nudge_x = 1,
nudge_y = 1, check_overlap = TRUE)`
x, y, label, alpha, angle, color, family, fontface,
hjust, lineheight, size, vjust



`e + geom_jitter(height = 2, width = 2)`
x, y, alpha, color, fill, shape, size



`e + geom_point()`
x, y, alpha, color, fill, shape, size, stroke



`e + geom_quantile()`
x, y, alpha, color, group, linetype, size, weight



`e + geom_rug(sides = "bl")`
x, y, alpha, color, linetype, size



`e + geom_smooth(method = lm)`
x, y, alpha, color, fill, group, linetype, size, weight



`e + geom_text(aes(label = cty), nudge_x = 1,
nudge_y = 1, check_overlap = TRUE)`
x, y, label, alpha, angle, color, family, fontface,
hjust, lineheight, size, vjust

X Discreta, Y Continua

`f <- ggplot(mpg, aes(class, hwy))`



`f + geom_col()`
x, y, alpha, color, fill, group, linetype, size



`f + geom_boxplot()`
x, y, lower, middle, upper, ymax, ymin, alpha,
color, fill, group, linetype, shape, size, weight



`f + geom_dotplot(binaxis = "y",
stackdir = "center")`
x, y, alpha, color, fill, group



`f + geom_violin(scale = "area")`
x, y, alpha, color, fill, group, linetype, size,
weight

X Discreta, Y Discreta

`g <- ggplot(diamonds, aes(cut, color))`



`g + geom_count()`
x, y, alpha, color, fill, shape, size, stroke

Distribución Bivariada Continua

`h <- ggplot(diamonds, aes(carat, price))`



`h + geom_bin2d(binwidth = c(0.25, 500))`
x, y, alpha, color, fill, linetype, size, weight



`h + geom_density2d()`
x, y, alpha, color, group, linetype, size



`h + geom_hex()`
x, y, alpha, color, fill, size

Función Continua

`i <- ggplot(economics, aes(date, unemploy))`



`i + geom_area()`
x, y, alpha, color, fill, linetype, size



`i + geom_line()`
x, y, alpha, color, group, linetype, size



`i + geom_step(direction = "hv")`
x, y, alpha, color, group, linetype, size

Visualizando el Error

`df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)`

`j <- ggplot(df, aes(grp, fit, ymin = fit-se, ymax = fit+se))`



`j + geom_crossbar(latten = 2)`
x, y, ymax, ymin, alpha, color, fill, group,
linetype, size



`j + geom_errorbar()`
x, ymax, ymin, alpha, color, group, linetype,
size, width (also `geom_errorbarh()`)



`j + geom_linerange()`
x, ymin, ymax, alpha, color, group, linetype, size



`j + geom_pointrange()`
x, y, ymin, ymax, alpha, color, fill, group,
linetype, shape, size

Mapas

`data <- data.frame(murder = USArrests$Murder,
state = tolower(rownames(USArrests)))`
`map <- map_data("state")`
`k <- ggplot(data, aes(fill = murder))`



`k + geom_map(aes(map_id = state), map = map) +
expand_limits(x = map$long, y = map$lat)`
map_id, alpha, color, fill, linetype, size

Trés Variables

`seals$z <- with(seals, sqrt(delta_long^2
+ delta_lat^2))`

`l <- ggplot(seals, aes(long, lat))`



`l + geom_raster(aes(fill = z),
hjust = 0.5, vjust = 0.5,
interpolate = FALSE)`
x, y, alpha, fill



`l + geom_tile(aes(fill = z))`
x, y, z, alpha, color, group,
linetype, size, weight

Argumentos

Traducción de argumentos comunes

label = etiqueta, angle = ángulo
size = tamaño, weight = peso
alpha = transparencia
fontface = tipo de letra
hjust = ajuste horizontal
lineheight = grosor de línea

ESTRUCTURA BASICA DE UN GRAFICO

```
ggplot(datos, aes(x, y)) +  
  geom_*
```

🔍 Componentes:

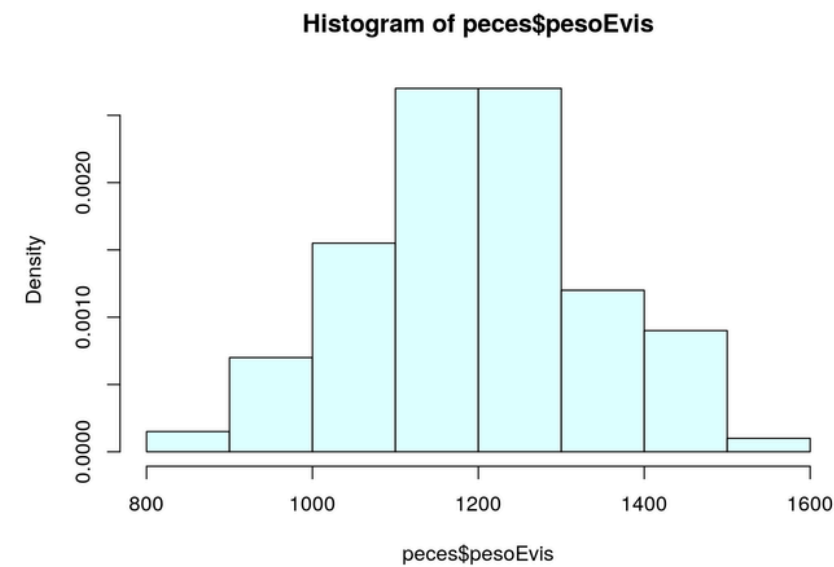
- ♦ datos → dataset
- ♦ aes() → variables (x, y, color)
- ♦ geom_* → tipo de gráfico

EJEMPLO:

```
ggplot(datos, aes(x = edad, y = biomarcador)) +  
  geom_point()
```

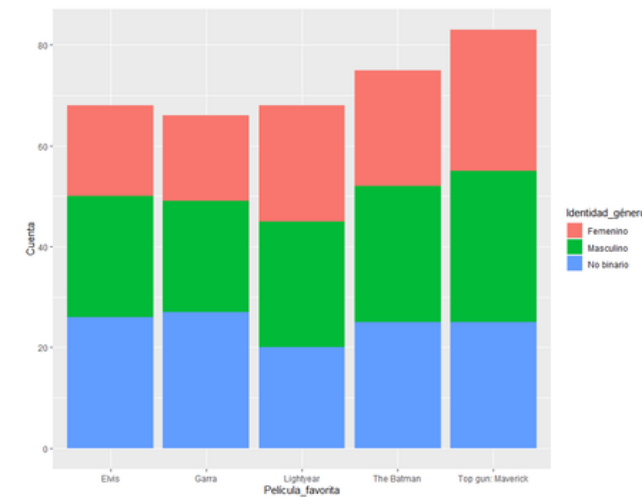
Relación entre dos variables continuas

GRÁFICOS MAS USADOS EN ARTÍCULOS CIENTÍFICOS



HISTOGRAMAS

Representa variables cuantitativas continuas. Distribución de frecuencia de un conjunto de datos numéricos continuos



GRAFICOS DE BARRAS

Representa variables categóricas o discretas. El eje x representa las categorías y el eje y representa la frecuencia con la que aparecen esas categorías.

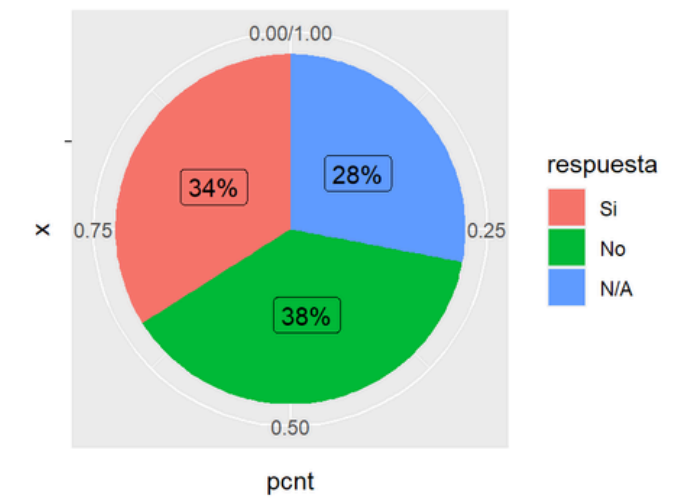


DIAGRAMA DE PASTEL

Representa proporciones o porcentajes de categorías. No son ideales para datos científicos. Son menos precisos que los gráficos de barras

GRÁFICOS MAS USADOS EN ARTÍCULOS CIENTÍFICOS

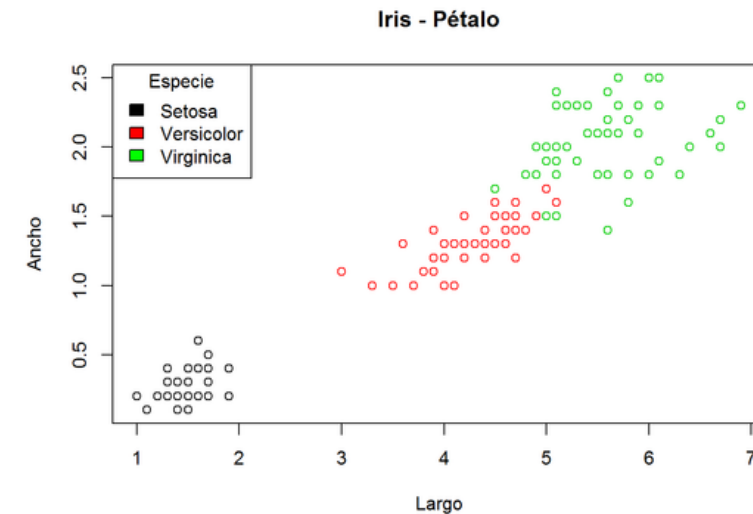


DIAGRAMA DE DISPERSIÓN

Muestra la relación entre dos variables cuantitativas. Cada punto en el gráfico representa una observación.

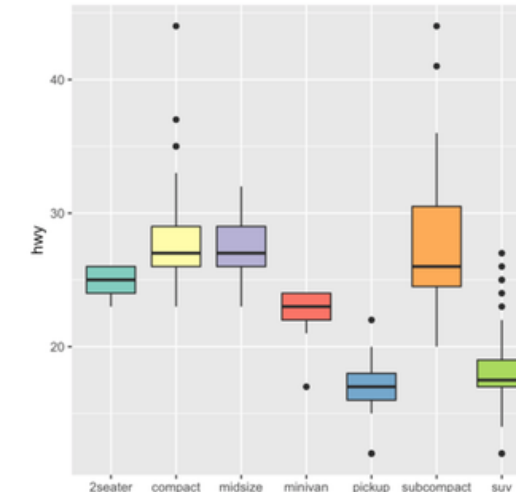
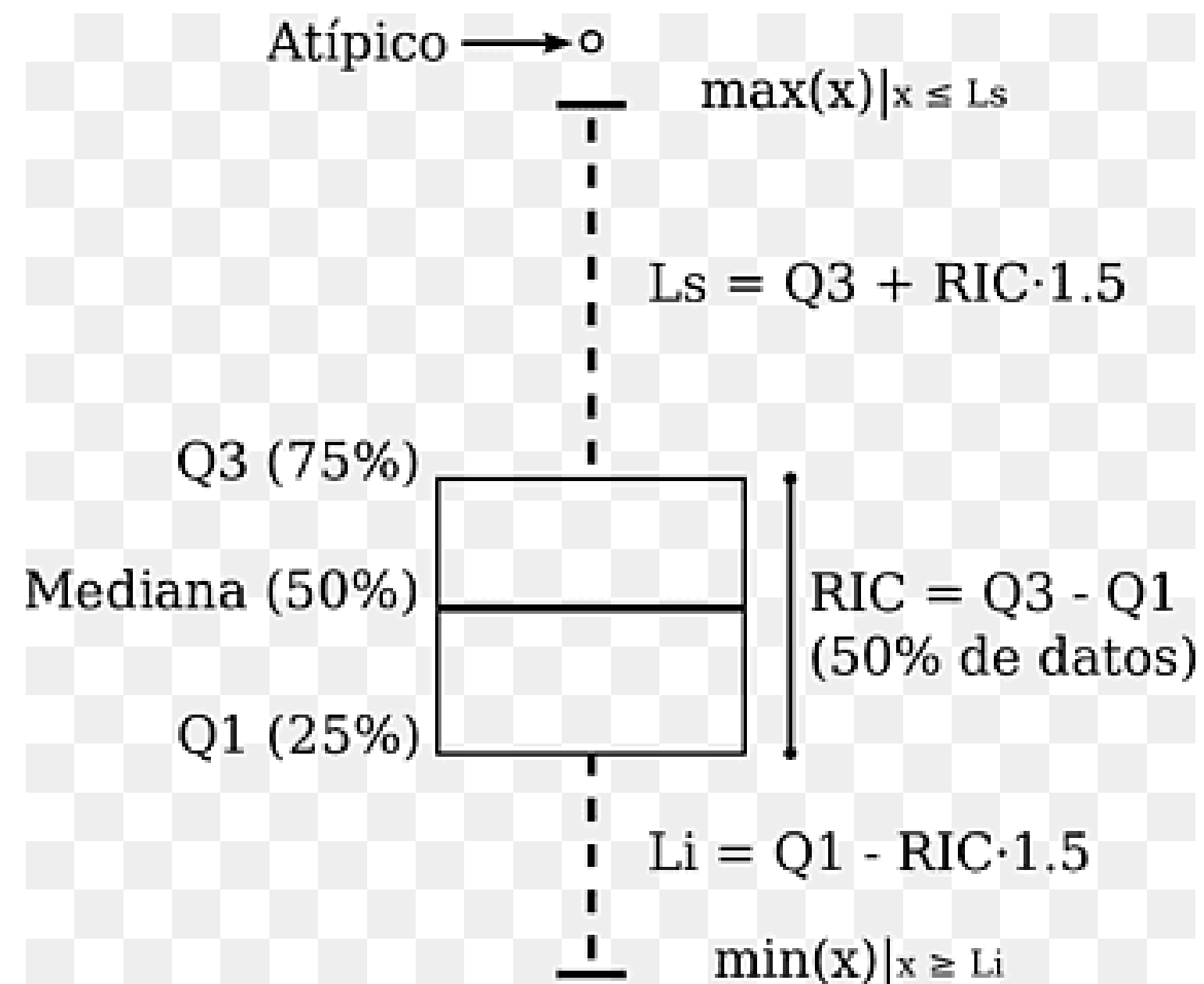


DIAGRAMA DE CAJA Y BIGOTES (BOXPLOT)

Resume la distribución de un conjunto de datos numéricos. Muestra la mediana, los cuartiles y posibles valores atípicos.

GRÁFICOS MAS USADOS EN ARTÍCULOS CIENTÍFICOS



GRÁFICOS MAS USADOS EN ARTÍCULOS CIENTÍFICOS

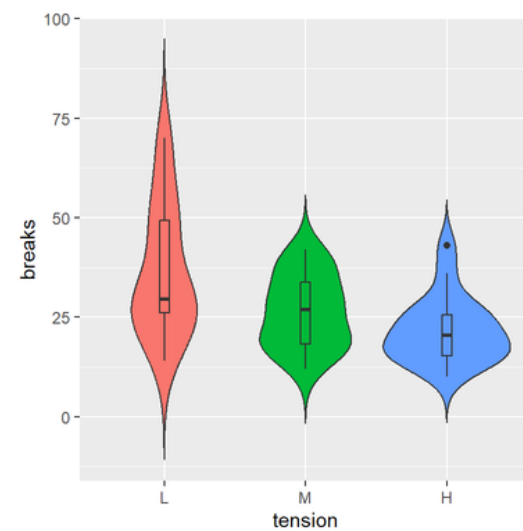
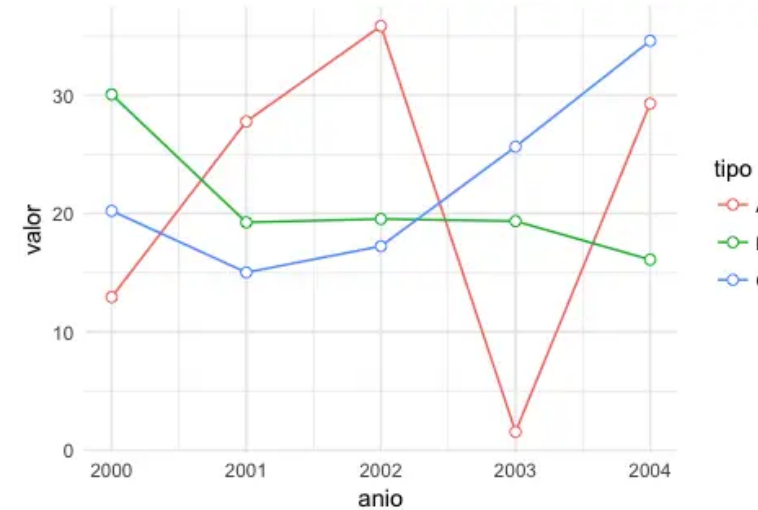


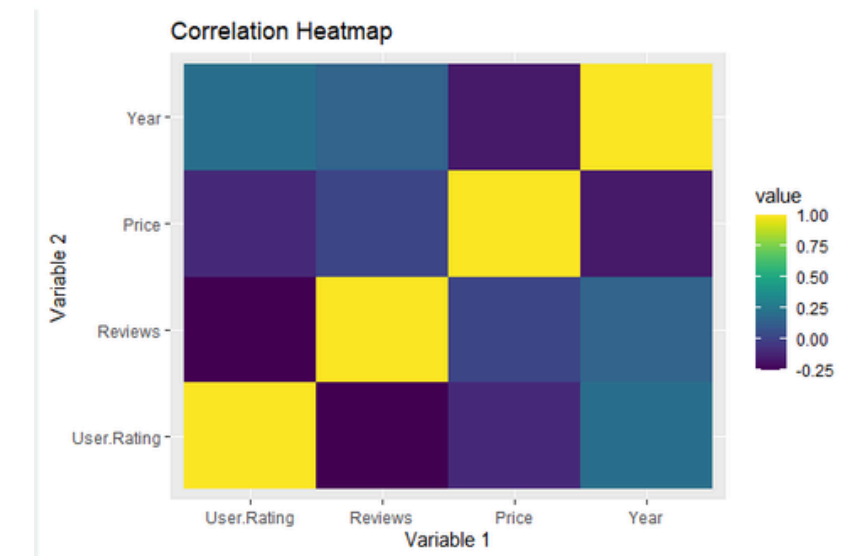
GRÁFICO DE VIOLIN

Similar al boxplot, pero muestra la forma de la distribución (Más ancho → más datos en ese valor. Más delgado → menos datos)



GRÁFICOS DE LÍNEAS

Cambios a lo largo del tiempo o condiciones



MAPA DE CALOR (HEATMAP)

Representa valores numéricos mediante colores, permitiendo identificar patrones y diferencias de forma visual.

ELEMENTOS ESCENCIALES

Etiquetas y títulos

```
labs(  
  title = "Relación edad vs biomarcador",  
  x = "Edad (años)",  
  y = "Biomarcador"  
)
```

ELEMENTOS ESCENCIALES

Leyendas y escalas

Leyendas

📌 Explican el significado de los elementos visuales

Escalas

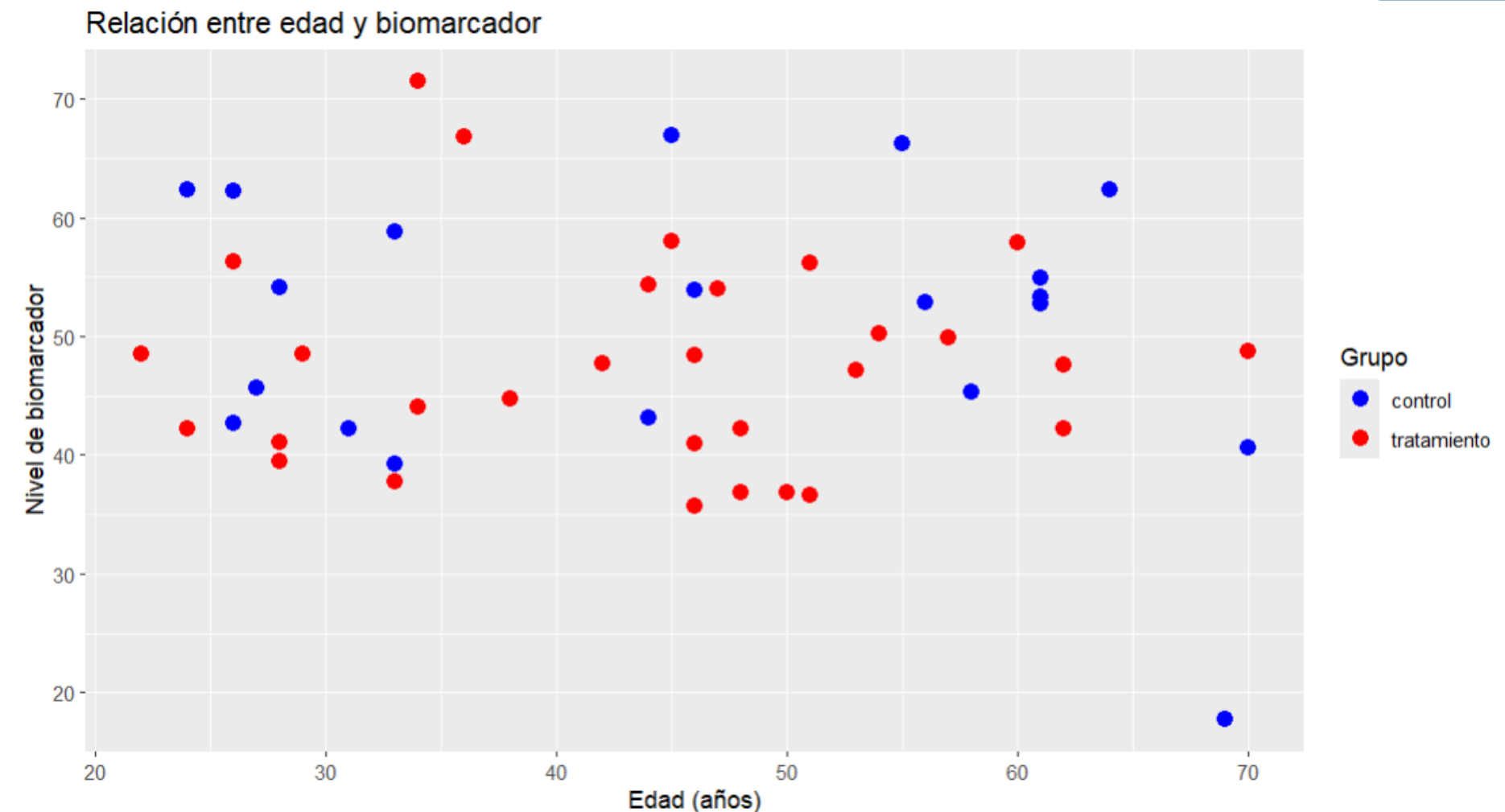
📌 Definen cómo se muestran los datos

```
library(ggplot2)

ggplot(datos, aes(x = edad, y = biomarcador, color = grupo)) +
  geom_point(size = 3) +

  # Etiquetas
  labs(
    title = "Relación entre edad y biomarcador",
    x = "Edad (años)",
    y = "Nivel de biomarcador",
    color = "Grupo" # 📌 Nombre de La Leyenda
  ) +

  # Escala de colores
  scale_color_manual(
    values = c("control" = "blue", "tratamiento" = "red")
  )
```



<https://r-charts.com/es/colores/>

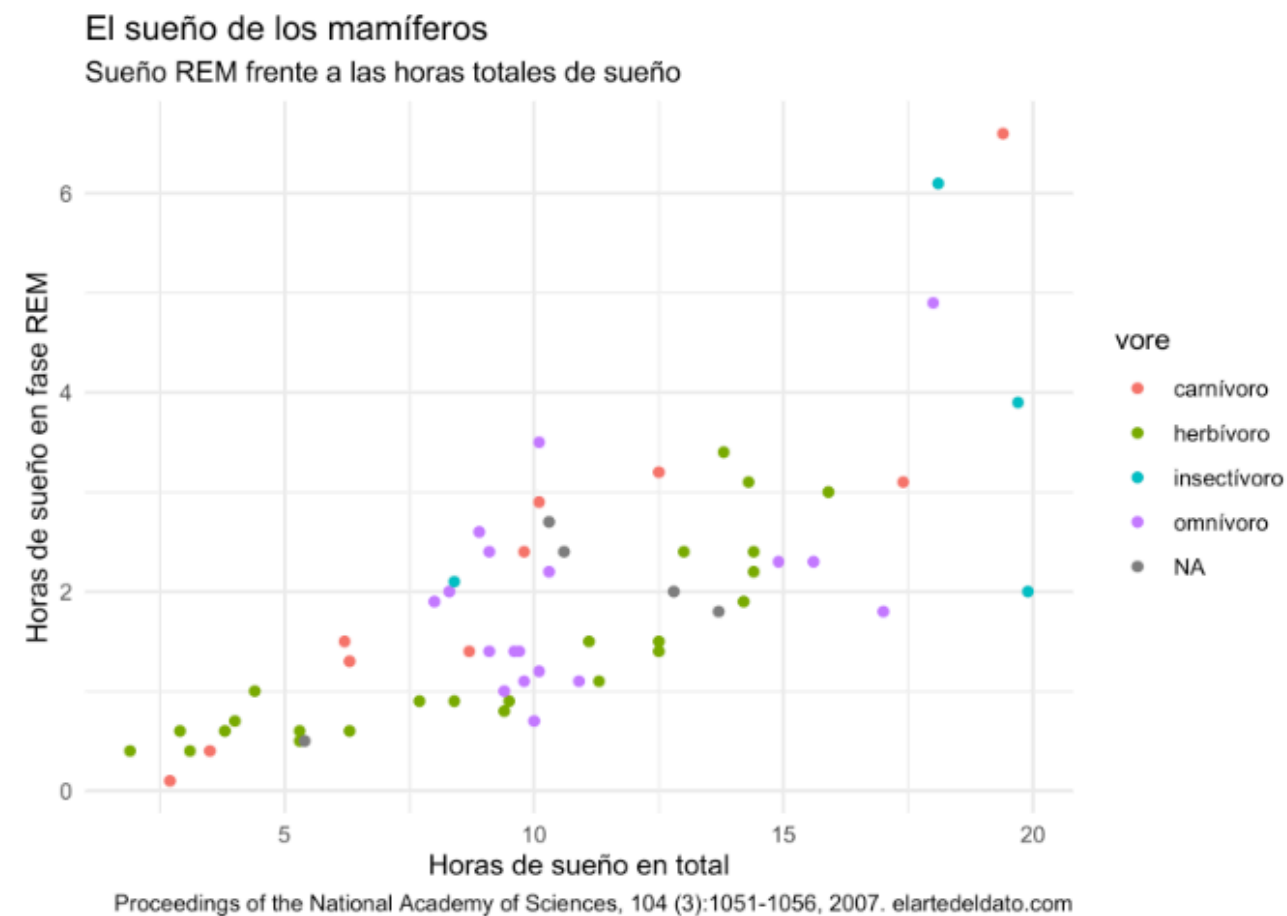
TEMAS EN GGPLOT2

🎨 Los temas son configuraciones que controlan la apariencia visual del gráfico, como el fondo, las líneas, las fuentes y la disposición de los elementos.

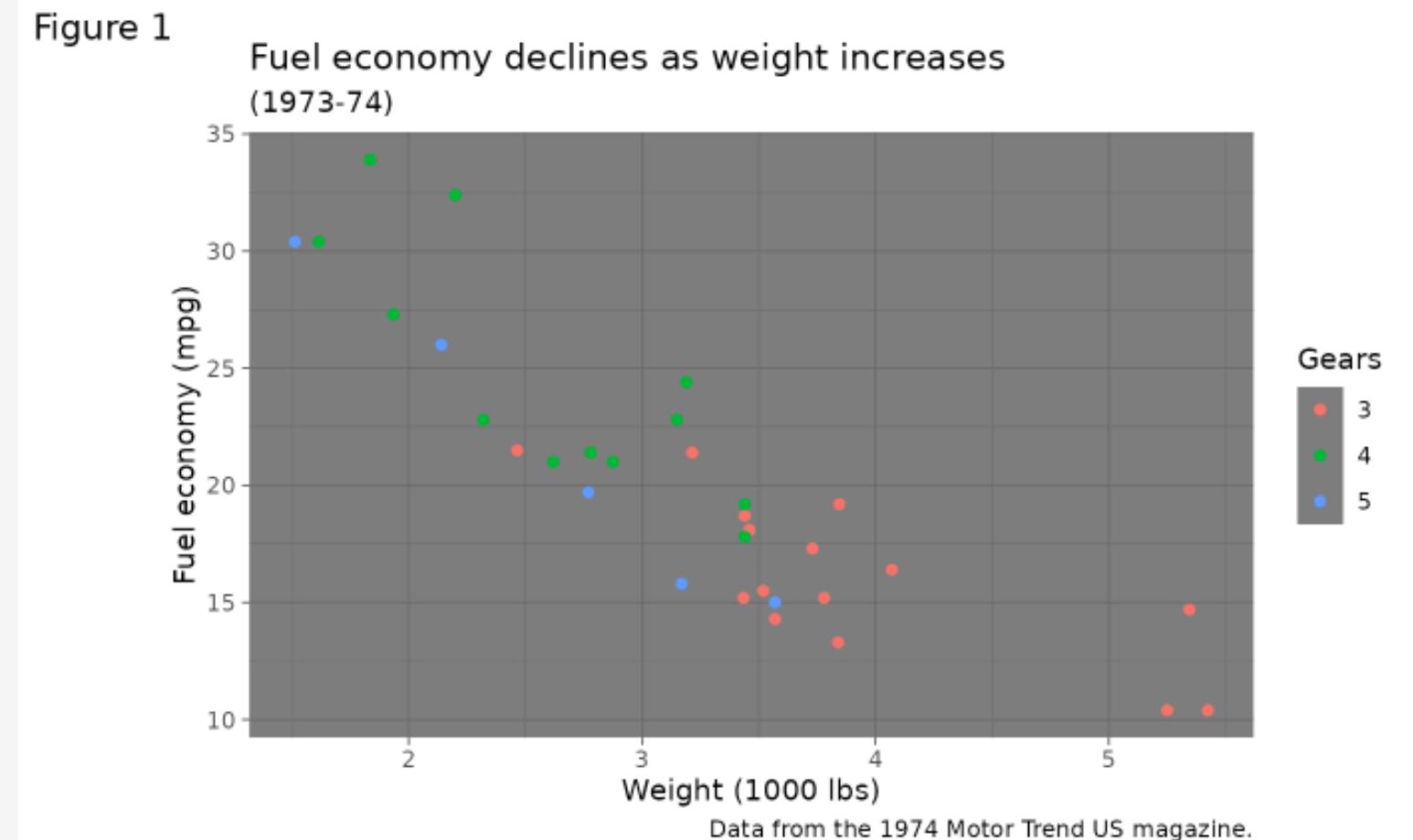
Tema	¿Para qué se utiliza?
<code>theme_classic()</code>	Gráficos limpios sin fondo ni cuadrícula, muy usados en artículos científicos
<code>theme_bw()</code>	Fondo blanco con cuadrícula, útil para resaltar valores y facilitar lectura
<code>theme_minimal()</code>	Diseño moderno y simple, elimina elementos innecesarios
<code>theme_light()</code>	Similar a minimal pero con líneas suaves, buena legibilidad
<code>theme_linedraw()</code>	Líneas más marcadas, útil para impresión en blanco y negro
<code>theme_dark()</code>	Fondo oscuro, más usado en presentaciones que en artículos
<code>theme_void()</code>	Elimina todos los elementos, útil para gráficos muy personalizados o mapas
<code>theme_gray()</code> (default)	Tema por defecto de ggplot2, no siempre ideal para publicación

TEMAS EN GGPLOT2

```
p + theme_minimal()
```



```
p1 + theme_dark()
```



<https://ggplot2.tidyverse.org/reference/ggtheme.html>

<https://elartedeldato.com/ggplot2/tema>

PALETAS DE COLORES

Son conjuntos de colores utilizados para representar visualmente los datos en un gráfico. Permiten diferenciar variables y mejorar la interpretación

1. Paletas categóricas

👉 Se usan cuando los datos son categorías (grupos)

✓ Ejemplo:

Control

Tratamiento

✓ Cada grupo tiene un color diferente

✨ Sirven para diferenciar grupos

```
aes(color = grupo)  
scale_color_manual(...)
```

2. Paletas continuas

👉 Se usan cuando los datos son numéricos

✓ Ejemplo:

Valores bajos → colores claros

Valores altos → colores intensos

✨ Sirven para mostrar gradientes o intensidad

```
aes(color = biomarcador)  
scale_color_viridis_c()
```

https://ggplot2.tidyverse.org/reference/scale_viridis.html

PATCHWORK Y COWPLOT

Ambos paquetes se usan para combinar varios gráficos en una sola figura.

Patchwork

👉 Combinar gráficos de forma muy sencilla e intuitiva

```
p1 + p2      # Lado a lado  
p1 / p2      # uno encima del otro  
(p1 | p2) / p3
```

Cowplot

👉 Combinar gráficos con más control y personalización

```
plot_grid(p1, p2, ncol = 2)
```

<https://patchwork.data-imaginist.com/>

<https://r-graph-gallery.com/package/cowplot.html>

EXPORTACIÓN DE GRÁFICOS

- 📁 Guardar en alta calidad
- ✓ Resolución $\geq$ 300 dpi
- ✓ Tamaño adecuado
- ✓ Tipografía legible

```
ggsave("figura1.png", dpi = 300, width = 6, height = 4)
```


¿QUÉ HACE UN GRÁFICO “PUBLICABLE”?

- ✓ Claro y fácil de interpretar
 - ✓ Mantener consistencia
 - ✓ Sin elementos innecesarios
- ✓ Elegir el gráfico correcto, según tipo de datos (dispersión, boxplot, etc.)
 - ✓ Etiquetas completas
 - ✓ Preferir estilos simples
- ✓ Evitar fondos oscuros, usar `theme_classic()` o `theme_bw()`
 - ✓ Evitar colores saturados
 - ✓ Usar paletas apropiadas (ej: viridis, escala de grises)
- ✓ Alta calidad (resolución). Formato según revista (PNG, TIFF, PDF)

“Si un lector entiende el gráfico sin leer el texto, entonces está bien hecho.”



MODULO 3

TABLAS CIENTÍFICAS Y CASOS PRÁCTICOS

TABLAS DESCRIPTIVAS CON DPLYR

- 📌 Resumen de variables
 - ✓ Media
 - ✓ Mediana
 - ✓ Desviación estándar
 - ✓ Conteos

```
datos %>%  
  group_by(grupo) %>%  
  summarise(  
    media = mean(biomarcador),  
    sd = sd(biomarcador)  
  )
```

TABLAS CON GT

-  Tablas modernas y elegantes
- ✓ Muy usadas en reportes
- ✓ Fácil personalización

```
library(gt)

tabla %>%
  gt()
```

TABLAS CON KABLEEXTRA

-  Ideal para documentos en HTML
- ✓ Tablas limpias
- ✓ Compatible con R Markdown

TABLAS DE MODELOS ESTADÍSTICOS



Presentar resultados de regresión

- ✓ Coeficientes
- ✓ p-values
- ✓ Intervalos de confianza

Herramientas principales:

- ◆ sjPlot → visualización de modelos

EXPORTAR TABLAS



Guardar tablas
para publicación

- ✓ Word
- ✓ HTML
- ✓ CSV



CLAVES PARA TABLAS CIENTÍFICAS

- ✓ Claridad
- ✓ Precisión
- ✓ Consistencia
- ✓ Formato adecuado
- ✓ Las tablas complementan los gráficos
- ✓ Permiten presentar resultados detallados

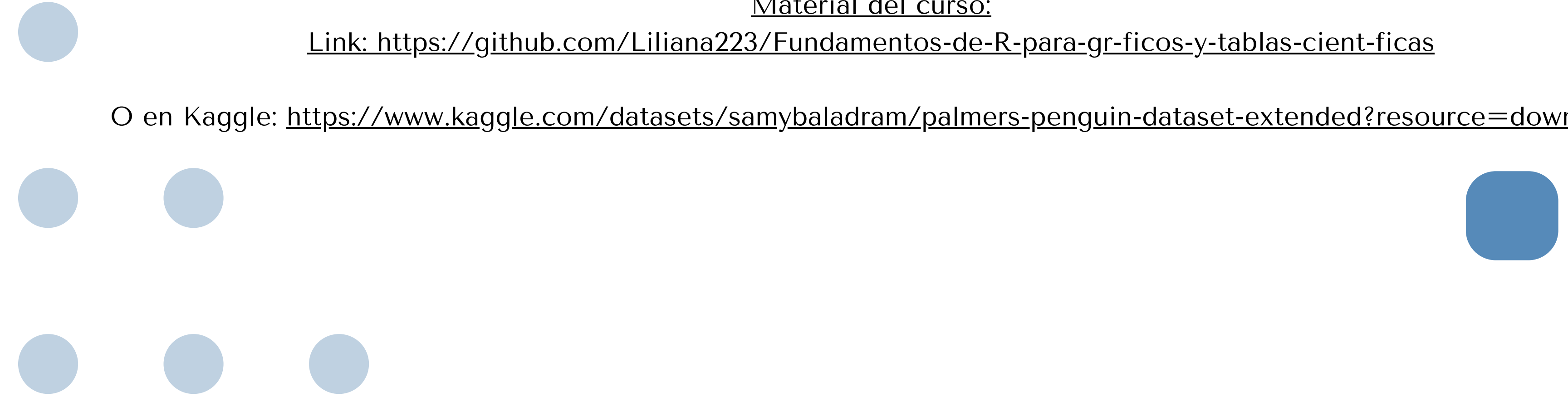


RETO DE PROGRAMACIÓN!!

Material del curso:

Link: <https://github.com/Liliana223/Fundamentos-de-R-para-gr-ficos-y-tablas-cient-ficas>

O en Kaggle: <https://www.kaggle.com/datasets/samybaladram/palmers-penguin-dataset-extended?resource=download>



RECURSOS ADICIONALES

dplyr: <https://nyu-cdsc.github.io/learningr/assets/data-transformation.pdf>

A ggplot2 Tutorial for Beautiful Plotting in R: <https://www.cedricscherer.com/2019/08/05/a-ggplot2-tutorial-for-beautiful-plotting-in-r/>

ggplot2: Elegant Graphics for Data Analysis (3e): <https://ggplot2-book.org/>

Fundamentos de Programación - BUCLES (Ciclos) EN PROGRAMACION: <https://www.youtube.com/watch?v=EbVT3XIf2TU>

Introducción a R. Bucles: <https://www.youtube.com/watch?v=w55TI908-Tw>

R for Data Science: <https://r4ds.had.co.nz/index.html>

R for Data Science Cookbook — Mark P. J. van der Loo & Yihui Xie (O'Reilly)

Líneas de referencia, segmentos, curvas y flechas en ggplot2: <https://r-charts.com/es/ggplot2/lineas-curvas/>



¡MUCHAS
GRACIAS!

